Local SSL for .test Domains

Date: 2026-03-16
Issue: #14

Overview

Local SSL for `.test` domains via a local Certificate Authority. After `outport setup`, every `.test` hostname automatically gets browser-trusted HTTPS with zero per-project configuration. The proxy terminates TLS — backends always receive plain HTTP on localhost.

Decisions

- **Local CA, not Let's Encrypt** — Let's Encrypt won't issue certs for IANA-reserved TLDs like `.test`. Local CA is the proven approach (used by mkcert, Portless, puma-dev, Laravel Valet).
- **Go `crypto/x509` + `crypto/ecdsa`** — stdlib, no external dependencies. Portless shells out to `openssl` (they're Node.js), but Go has first-class support.
- **SNI callback for on-demand certs** — per-hostname certs generated lazily as new hostnames appear (worktrees, new projects). No pre-generation needed.
- **Folded into `outport setup`** — no separate `outport trust` command. Setup already does one-time privileged operations (resolver file, LaunchAgent). Adding the CA to the system trust store fits that pattern. Fewer commands to remember.
- **Automatic HTTPS** — no per-service opt-in. If SSL is set up and a service has a hostname, it gets HTTPS. `http://` redirects to `https://`. The `protocol` field in `.outport.yml` means "this service has a web hostname," not HTTP vs HTTPS.
- **Dual-port (80 + 443)** — standard ports. Browsers expect HTTPS on 443. The LaunchAgent plist declares two named sockets. Byte-peeking (serving both protocols on one port) only works for non-standard ports and would break `https://myapp.test` (which hits 443 by default).
- **No HSTS headers** — the proxy does not send `Strict-Transport-Security`. For local dev, HSTS can cause problems if SSL is later removed (browsers cache the directive and refuse plain HTTP).

XDG Directory Layout

Move all Outport data to proper XDG locations:

Path	Contents	Rationale
`~/.config/outport/`	Future global config	Portable, dotfiles-safe
`~/.local/share/outport/registry.json`	Registry	Machine-specific (contains absolute paths), persistent
`~/.local/share/outport/ca-key.pem`	CA private key (0600)	Persistent, painful to regenerate (requires re-trusting)
`~/.local/share/outport/ca-cert.pem`	CA certificate	Persistent, installed in system trust store
`~/.cache/outport/certs/`	Per-hostname server certs	Regenerable on demand, signed by CA

The registry moves from `~/.config/outport/registry.json` to `~/.local/share/outport/registry.json`. All code referencing the old path updates.

Certificate Authority

- Generated during `outport setup` using Go `crypto/x509` + `crypto/ecdsa`
- EC P-256 key, 10-year validity
- CA key written with 0600 permissions (owner-only read/write)
- Subject: `O=Outport Dev CA, CN=Outport Dev CA`
- Stored at `~/.local/share/outport/ca-key.pem` and `ca-cert.pem`
- Added to macOS login keychain trust store via: `security add-trusted-cert -r trustRoot -k ~/Library/Keychains/login.keychain-db ca-cert.pem` (prompts for login keychain password via macOS GUI dialog, not a terminal sudo prompt)
- `outport teardown` removes the CA from the trust store and deletes the files

Server Certificates

- Generated on demand via TLS SNI callback — when the proxy receives a request for `myapp.test`, it checks the cache, generates a cert if needed
- Signed by the local CA, EC P-256 key, 1-year validity
- Key files written with 0600 permissions
- Must include `SubjectAlternativeName` (SAN) with the hostname — CN alone is insufficient (Chrome requirement since Chrome 58)
- Cached to `~/.cache/outport/certs/{hostname}.pem` and `{hostname}-key.pem`

- Memory cache in the daemon (synchronized with `sync.RWMutex`) so disk isn't hit on every request
- Regenerate if within 7 days of expiry, or if the cert was not signed by the current CA (handles CA regeneration after teardown+setup)
- Each hostname gets its own cert, generated lazily — no wildcards needed

Daemon Changes

Dual Socket Activation

The LaunchAgent plist declares two named sockets: `HTTPSocket` (port 80) and `HTTPSSocket` (port 443). Both use `SockNodeName: 127.0.0.1` (localhost only). The daemon calls `launchd.Activate("HTTPSocket")` and `launchd.Activate("HTTPSSocket")` to receive both file descriptors. The existing `activateLaunchdSocket()` in `daemon_darwin.go` (which currently calls `launchd.Activate("Socket")`) is replaced by two calls with the new socket names.

`DaemonConfig` expands from a single `Listener net.Listener` field to `HTTPListener net.Listener` and `HTTPSListener net.Listener`. `daemon.Run()` starts two `http.Server` instances: one for HTTP (redirect) and one for HTTPS (TLS proxy).

Plist socket section:

```
<key>Sockets</key>
<dict>
    <key>HTTPSocket</key>
    <dict>
        <key>SockNodeName</key>
        <string>127.0.0.1</string>
        <key>SockServiceName</key>
        <string>80</string>
    </dict>
    <key>HTTPSSocket</key>
    <dict>
        <key>SockNodeName</key>
        <string>127.0.0.1</string>
        <key>SockServiceName</key>
        <string>443</string>
    </dict>
</dict>
```

TLS Listener

The daemon creates an `http.Server` with `tls.Config` on the `HTTPSSocket` listener. The `GetCertificate` callback implements the SNI logic:

1. Check memory cache for the requested hostname
2. If miss, check disk cache (`~/.cache/outport/certs/`)
3. If miss, expired (within 7 days), or signed by a different CA than the current one, generate a new cert signed by the CA
4. Cache to memory and disk, return the cert

HTTP → HTTPS Redirect

The port 80 handler redirects all requests to `https://{host}{path}` with a 307 status (temporary redirect). 307 preserves the HTTP method and avoids browser caching issues — if SSL is later torn down, browsers won't be stuck trying HTTPS.

Proxy

After TLS termination, requests are forwarded to `http://localhost:{port}` exactly as today. The reverse proxy logic is unchanged. Backends see plain HTTP — they never know TLS is involved. WebSocket proxying continues to work transparently.

`outport setup` Changes

`outport setup` adds these steps after the existing resolver file and plist steps:

1. Check that ports 80 and 443 are both available (extends existing port 80 check)
2. Generate CA key + cert if not already present at `~/.local/share/outport/`
3. Add CA to macOS login keychain trust store (prompts for login keychain password via macOS GUI dialog — this is separate from the `sudo` terminal prompt for the resolver file). If the user cancels the dialog, setup fails with a clear message explaining the CA must be trusted for HTTPS to work.
4. Updated plist now declares both `HTTPSocket` (port 80) and `HTTPSSocket` (port 443)

Idempotent: if the CA already exists and is trusted, these steps are skipped.

`outport teardown` Changes

`outport teardown` adds:

1. Remove CA from login keychain trust store (`security remove-trusted-cert ca-cert.pem`)
2. Delete CA files from `~/.local/share/outport/`

3. Delete cached server certs from `~/.cache/outport/certs/`

`outport apply` Changes

- Checks if CA exists at `~/.local/share/outport/ca-cert.pem`
- If present: `buildTemplateVars` uses `https` as the scheme for `${service.url}` regardless of the `protocol` field value. Example: `${rails.url}` → `https://myapp.test`
- If absent: `buildTemplateVars` uses the `protocol` field as today (`http://myapp.test`)
- `${service.url:direct}` is unaffected — always `http://localhost:{port}`
- No changes to `.outport.yml` format — the `protocol` field continues to mean "this service has a web hostname"

`outport open` Changes

- Checks if CA exists at `~/.local/share/outport/ca-cert.pem`
- If present: opens `https://{hostname}` regardless of the `protocol` field value
- If absent: opens `{protocol}://{hostname}` as today

`--json` Output

`outport setup` JSON output adds:

- `ca_generated` (bool) — whether a new CA was generated
- `ca_trusted` (bool) — whether the CA was added to the trust store

`outport teardown` JSON output adds:

- `ca_removed` (bool) — whether the CA was removed from the trust store
- `certs_cleaned` (bool) — whether cached certs were deleted

What Doesn't Change

- **`.outport.yml` format** — no new fields needed for SSL
- **Backend apps** — always receive plain HTTP on localhost
- **`${service.url:direct}`** — always produces `http://localhost:{port}` (direct bypass)
- **DNS** — still resolves `*.test` to `127.0.0.1` on port 15353
- **Registry structure** — same fields (ports, hostnames, protocols)

New Internal Package

`internal/certmanager` — handles CA generation, server cert generation, disk/memory caching, expiry checks, and CA signature validation. Used by the daemon's TLS listener and by `setup` / `teardown` commands for CA lifecycle. Exports path functions (`CAKeyPath()` , `CACertPath()` , `CertCacheDir()`) so that `cmd/apply.go` , `cmd/open.go` , and other consumers don't hardcode paths.

Platform-Specific Notes (macOS)

- CA trust: `security add-trusted-cert -r trustRoot -k ~/Library/Keychains/login.keychain-db ca-cert.pem` (GUI password prompt)
- CA untrust: `security remove-trusted-cert ca-cert.pem`
- Ports 80 and 443 via launchd socket activation — daemon itself doesn't need root
- Linux support (future): `update-ca-certificates` (Debian) / `update-ca-trust` (Fedora/Arch)